

Count Array Pairs Divisible by K

Target Companies: Amazon, PayPal

Problem Statement

Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs (i, j) such that $0 \leq i < j < n$ and $(\text{nums}[i] \times \text{nums}[j]) \% k == 0$.

Brute Force Approach

The naive approach is to try every possible pair (i, j) using two nested loops and check if their product satisfies the divisibility condition.

Time Complexity	$O(n^2)$
Space Complexity	$O(1)$

Optimal Approach

Instead of directly validating $(\text{nums}[i] \times \text{nums}[j]) \% k == 0$, we can analyze the specific part of each number that contributes to making the product divisible by `k`.

Essentially, for the product to be a multiple of `k`, the combined prime factors from both numbers must cover all the prime factors of `k`. Therefore, instead of tracking the full numbers, we only care about their greatest common divisor (GCD) with respect to `k`:

$$(\text{gcd}(\text{nums}[i], k) \times \text{gcd}(\text{nums}[j], k)) \% k == 0$$

While scanning the array from left to right, we can efficiently store and look up the useful parts of each visited number. We maintain a frequency map of the GCD values processed so far:

$$g = \text{gcd}(\text{num}, k)$$

C++ Implementation

```
class Solution {
public:
    long long countPairs(vector<int>& nums, int k) {

        int n = nums.size();
        unordered_map<int, long long> mp;

        long long ans = 0;

        for (int i = 0; i < n; i++) {
            int gcd = __gcd(nums[i], k);
            for (auto it = mp.begin(); it != mp.end(); it++) {
                int othergcd = it->first;
                int freq = it->second;

                if ((1LL * gcd * othergcd) % k != 0) continue;
                ans += freq;
            }

            mp[gcd]++;
        }

        return ans;
    }
};
```

Document compiled on July 7, 2026
Authored by: **Arpit Raj** | LNMIIT Jaipur